

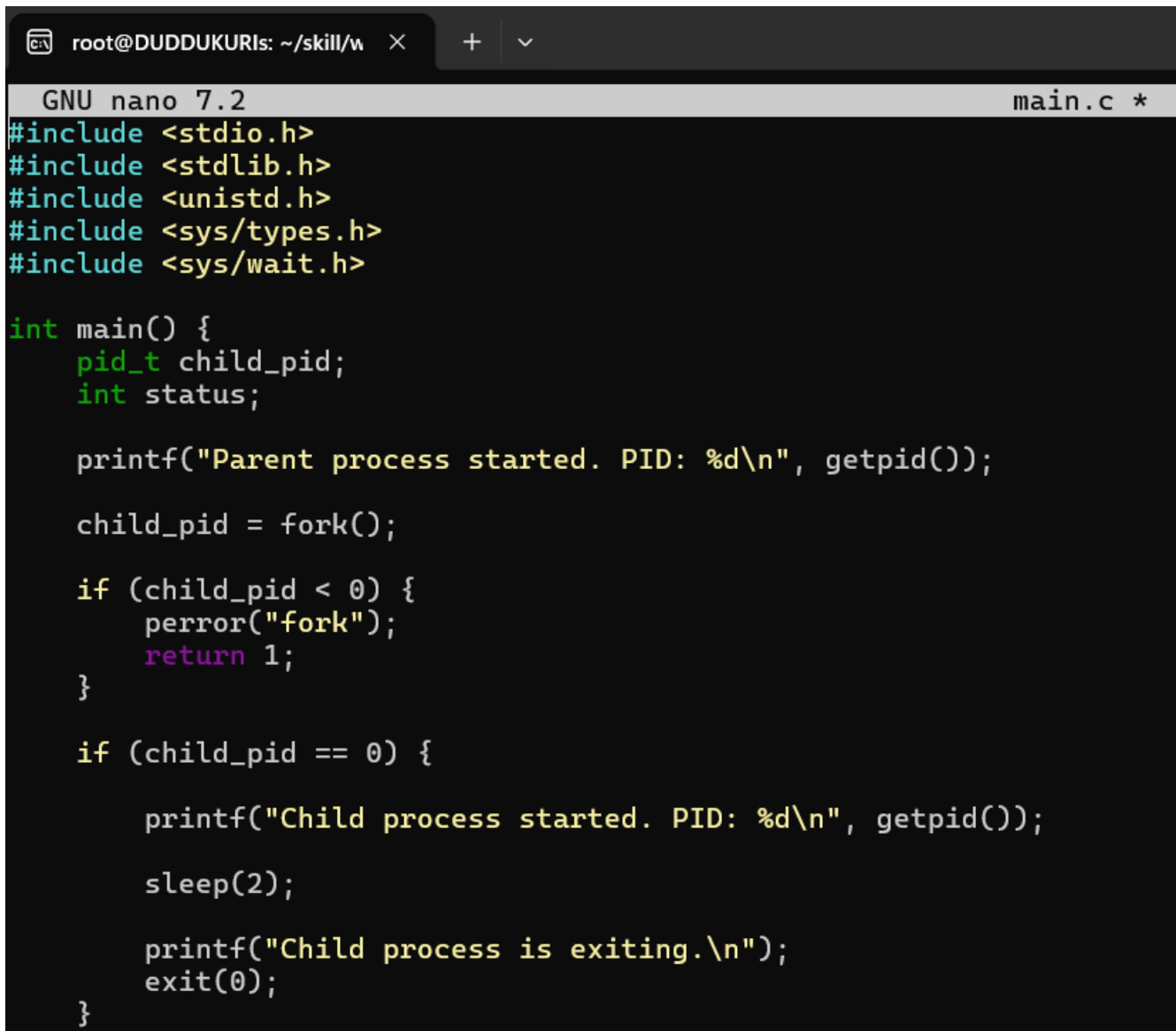
Week 4

SESSION-7

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

To create a child process, synchronize the parent with the child using waitpid(), monitor the child process, check normal termination, retrieve the exit status, handle signal termination, and test process synchronization.

CODE:



```
root@DUDDUKURIs: ~/skill/w × + ∨
GNU nano 7.2 main.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t child_pid;
    int status;

    printf("Parent process started. PID: %d\n", getpid());

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    if (child_pid == 0) {

        printf("Child process started. PID: %d\n", getpid());

        sleep(2);

        printf("Child process is exiting.\n");
        exit(0);
    }
}
```

```

    printf("Parent process is waiting for child PID: %d\n", child_pid);

    if (waitpid(child_pid, &status, 0) == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Child process terminated normally.\n");
        printf("Child exit status: %d\n", WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status)) {
        printf("Child process terminated by a signal.\n");
        printf("Signal number: %d\n", WTERMSIG(status));
    }

    printf("Parent process completed.\n");

    return 0;
}

```

COMPILE AND RUN:

```

root@DUDDUKURIs: ~/skill/w × + ∨
root@DUDDUKURIs:~# cd skill/week-7
root@DUDDUKURIs:~/skill/week-7# nano main.c
root@DUDDUKURIs:~/skill/week-7# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-7# ./main

```

OUTPUT:

```

root@DUDDUKURIs:~/skill/week-7# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-7# ./main
Parent process started. PID: 498
Parent process is waiting for child PID: 499
Child process started. PID: 499
Child process is exiting.
Child process terminated normally.
Child exit status: 0
Parent process completed.
root@DUDDUKURIs:~/skill/week-7# |

```

Task 2: PATH Variable Retrieval and Command Resolution

To retrieve the PATH variable, parse its search directories, locate executable files, verify execute permissions, handle missing commands, and test command resolution.

CODE:

```
root@DUDDUKURIs: ~/skill/vn × + v
GNU nano 7.2 main2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char *path;
    char *path_copy;
    char *directory;
    char full_path[PATH_MAX];
    char command[100];
    int found = 0;

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable is not set.\n");
        return 1;
    }

    printf("Enter command: ");
    scanf("%99s", command);

    path_copy = malloc(strlen(path) + 1);

    if (path_copy == NULL) {
        perror("malloc");
        return 1;
    }

    strcpy(path_copy, path);

    directory = strtok(path_copy, ":");

    while (directory != NULL) {

        snprintf(full_path, sizeof(full_path),
                 "%s/%s", directory, command);
```

```

        if (access(full_path, X_OK) == 0) {
            printf("Command found: %s\n", full_path);
            found = 1;
            break;
        }

        directory = strtok(NULL, ":");
    }

    if (!found) {
        printf("Command not found: %s\n", command);
    }

    free(path_copy);

    return 0;
}

```

OUTPUT:

```

root@DUDDUKURIs:~/skill/week-7# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-7# ./main2
Enter command: ls
Command found: /usr/bin/ls
root@DUDDUKURIs:~/skill/week-7# ./main2
Enter command: hello
Command not found: hello
root@DUDDUKURIs:~/skill/week-7# |

```

SESSION-8

Task 1: Variable Reference Detection and Value Expansion

To detect variable references, expand environment variable values, handle undefined variables, update tokens after expansion, support nested variables, and test expansion logic.

CODE:

```
root@DUDDUKURIs: ~/skill/w × + v
GNU nano 7.2 main.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1000
#define MAX_OUTPUT 2000

void expand_variables(const char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {

        if (input[i] == '$') {
            char variable[100];
            int k = 0;

            i++;

            if (input[i] == '{') {
                i++;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99) {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;
            }

            else {
                while (input[i] != '\0' &&
                    (isalnum((unsigned char)input[i]) ||
                    input[i] == '_') &&
                    k < 99) {
                    variable[k++] = input[i++];
                }
            }
        }
    }
}
```

```

        variable[k] = '\0';
    }

    if (k == 0) {
        output[j++] = '$';
        continue;
    }

    char *value = getenv(variable);

    if (value != NULL) {
        int v = 0;

        while (value[v] != '\0' &&
                j < MAX_OUTPUT - 1) {
            output[j++] = value[v++];
        }
    }
    else {
        printf("Undefined variable: %s\n", variable);
    }
}
else {
    output[j++] = input[i++];
}

```

```

    output[j] = '\0';
}

int main() {
    char input[MAX_INPUT];
    char expanded[MAX_OUTPUT];

    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal: %s\n", input);

    expand_variables(input, expanded);

    printf("Expanded: %s\n", expanded);

    return 0;
}

```

OUTPUT:

```
root@DUDDUKURIs: ~/skill/w × + v
root@DUDDUKURIs:~# cd skil/week-8
-bash: cd: skil/week-8: No such file or directory
root@DUDDUKURIs:~# cd skill/week-8
root@DUDDUKURIs:~/skill/week-8# nano main.c
root@DUDDUKURIs:~/skill/week-8# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-8# ./main
Enter a string: hello$world

Original: hello$world
Undefined variable: world
Expanded: hello
root@DUDDUKURIs:~/skill/week-8# |
```

Task 2: Built-in Identification, Dispatch Table and In-Process Execution

To identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test dispatch logic.

CODE:

```
root@DUDDUKURIs: ~/skill/w × + v
GNU nano 7.2 main2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

void builtin_pwd(void);
void builtin_cd(char *argument);
void builtin_echo(char *argument);
void builtin_exit(void);

struct builtin {
    char *name;
    void (*function)(void);
};

void builtin_pwd(void) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

void builtin_cd(char *argument) {
    if (argument == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    void builtin_echo(char *argument) {
        if (argument != NULL) {
            printf("%s\n", argument);
        } else {
            printf("\n");
        }
    }
}
```



```

void builtin_exit(void) {
    printf("Exiting shell...\n");
    exit(0);
}

struct builtin builtins[] = {
    {"pwd", NULL},
    {"cd", NULL},
    {"echo", NULL},
    {"exit", builtin_exit}
};

int main() {
    char input[MAX_INPUT];
    char *command;
    char *argument;
    int found;

    printf("Simple Built-in Command Shell\n");
    printf("Available commands: pwd, cd, echo, exit\n");

    while (1) {
        printf("shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            continue;
        }
    }

```

```

    command = strtok(input, " ");
    argument = strtok(NULL, "");

    found = 0;

    for (int i = 0; i < 4; i++) {
        if (strcmp(command, builtins[i].name) == 0) {
            found = 1;
            if (strcmp(command, "pwd") == 0) {
                builtin_pwd();
            }
        }
    }

```

```

        else if (strcmp(command, "cd") == 0) {
            builtin_cd(argument);
        }
        else if (strcmp(command, "echo") == 0) {
            builtin_echo(argument);
        }
        else if (strcmp(command, "exit") == 0) {
            builtins[i].function();
        }

        break;
    }
}

if (!found) {
    printf("Invalid command: %s\n", command);
}

return 0;
}

```

OUTPUT:

```

root@DUDDUKURIs: ~/skill/w × + ▾
root@DUDDUKURIs:~# cd skill/week-8
root@DUDDUKURIs:~/skill/week-8# nano main2.c
root@DUDDUKURIs:~/skill/week-8# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-8# ./main2
Simple Built-in Command Shell
Available commands: pwd, cd, echo, exit
shell> cd ..
shell> pwd
/root/skill
shell> hello
Invalid command: hello
shell> exit
Exiting shell...
root@DUDDUKURIs:~/skill/week-8# |

```